

Ambiente de trabajo con Agentes de IA — Fase de programación

Herramienta utilizada

Claude Code (Anthropic), un agente de IA de línea de comandos con acceso directo al repositorio: puede leer y escribir archivos, ejecutar comandos de shell (`npm`, `git`, `psql`, `curl`), y probar cambios contra un navegador real. No es autocompletado dentro de un editor: ejecuta el ciclo completo — leer código existente, escribir el cambio, correrlo contra la base de datos o el navegador, y corregir si falla — antes de entregarlo como terminado.

Reglas de contexto por repositorio (`CLAUDE.md`)

Cada repositorio (Barber-Kong y Barber-Kong-Frontend) tiene un archivo `CLAUDE.md` en la raíz que el agente lee automáticamente al iniciar. Define las reglas duras del proyecto para que la IA no las tenga que "adivinar" ni el equipo tenga que repetirlas en cada sesión:

- Sin ORM: todo acceso a datos es SQL parametrizado a mano vía `pg`.
- Cada endpoint valida el rol en el backend, nunca solo en el frontend.
- IDs UUID generados en la aplicación, no en la base de datos.
- Todo error pasa por el `errorHandler` central — nada de `console.log` como manejo de errores.
- Arquitectura por módulos de dominio (`routes` → `controller` → `service` → `repository` → `schema`), un directorio por módulo, sin capas técnicas a nivel raíz.
- Migraciones versionadas en `resources/db/migrations/`, nunca editar una ya aplicada.

Este archivo se actualiza cada vez que el equipo toma una decisión de arquitectura nueva (por ejemplo, al introducir las migraciones versionadas), de modo que quede como la fuente de verdad para cualquier sesión futura del agente, no solo como una nota de una conversación.

Flujo de trabajo (control de versiones)

- El agente nunca trabaja directo sobre `main` o `develop`: cada tarea se hace en una rama `feature/`, `fix/` o `docs/` nueva, creada a partir de `develop` actualizado.
- El agente **no hace** push **ni abre Pull Requests sin autorización explícita** del integrante que lo está usando en ese momento — solo prepara el commit localmente.
- Cada cambio de un integrante llega al otro vía Pull Request revisado y fusionado a `develop` (no por copia manual de archivos), así que antes de iniciar una tarea nueva el agente sincroniza la rama de trabajo contra el `develop` más reciente.
- Operaciones destructivas (`git reset --hard`, sobrescribir archivos con datos reales, borrar ramas) requieren confirmación explícita del integrante antes de ejecutarse.